МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки і комп’ютерних технологій

Звіт

про виконання лабораторної роботи №10

з курсу «Вибрані розділи науки про дані»

«Система підказок на основі онтології та тестові набори для малої мовної моделі»

Виконав:
студент групи ФЕІм-13с
Шевчук Максим Вікторович

Перевірив:

Львів 2026

Тема та мета роботи

Розроблено систему зіставлення опису запиту з класами онтології енергетичних даних. Замість великої мовної моделі використано вже завантажену Qwen3-0.6B з ЛР7. Мета — порівняти типи й версії підказок, перевірити динамічне використання онтології, виміряти якість та продемонструвати захист від ін’єкцій.

Експерименти виконано локально. Синтетичні еталонні мітки підготовлено за участі Codex; дві незалежні людські розмітки ще не отримано. Тому к Коена не визначено, а відповідний пункт результату 3 не можна вважати завершеним. Решту числових результатів наведено за реальними журналами запуску.

Початкові дані та середовище

Файл ontology.ttl — незмінена копія asserted-онтології з ЛР9. Для семи вибраних класів додано окремий шар skos:altLabel у data/synonyms.ttl. Початкові твердження ЛР9 не змінювалися. Завдання вузьке: визначити явно названий тип графіка, моделі, лічильник або часовий ряд; система не прогнозує споживання і не керує обладнанням.

Модель має 596,049,920 параметрів. Використано CPU, float32, чотири потоки, seed 42, enable_thinking=False. do_sample=False виконує жадібну генерацію — еквівалент temperature=0. Ліміт відповіді — 100 нових токенів; вхід моделі — до 8192 токенів. Python-середовище і ваги повторно використано з ЛР7.

Результат 1 Чотири типи підказок

Вихід — JSON з полями status і classes, без додаткових полів. Для status=ok потрібен один клас, для ambiguous — щонайменше два, для unknown або refused — порожня множина. Доступні класи: LineChart, Heatmap, RegressionModel, ClassificationModel, ClusteringModel, Meter, TimeSeries. JSON перевіряється jsonschema, після чого перевіряються існування класів та узгодженість статусу.

Таблиця 1 — Реалізовані типи підказок

Тип	**Відмінність**
Інструктивна	Пряме завдання, перелік міток і формат JSON
Рольова	Роль куратора онтології енергетичної візуалізації
Контекстна	Дані зі SPARQL і синоніми виділених понять
Few-shot	Три приклади: лінійний графік, теплова карта, запит поза темою

Таблиця 2 — Демонстраційні запуски

Тип	**Запитів**	**Валідних відповідей**
instruction	5	5
role	5	5
context	5	5
fewshot	5	5

Для кожного типу використано однакові п’ять запитів про різні класи. Повні підказки й сирі відповіді збережено в results/templates_*.json. Така демонстрація перевіряє працездатність форматів, але не замінює оцінювання на незалежних даних.

Результат 2 Динамічні підказки з онтології

Послідовність етапів і повторну перевірку показано на рис. 1.

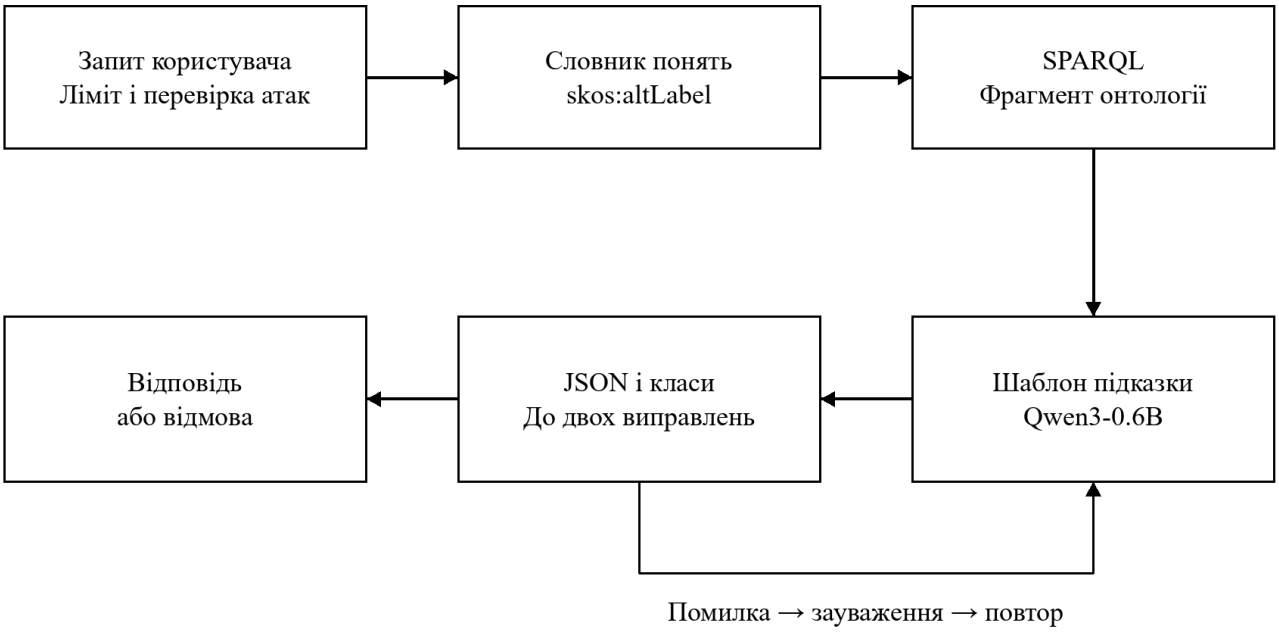


Рис. 1 — Система онтологічних підказок

Запит нормалізується Unicode NFKC і casefold; словник skos:altLabel дає множину понять. Межі слів зменшують помилкові збіги всередині інших слів. SPARQL VALUES містить лише довірені IRI класів. Запит повертає мітки, коментарі, батьківські класи, властивості з domain/range та доступні OWL-обмеження. Відсутність обмеження не замінюється вигаданим правилом.

Отриманий фрагмент і словник підставляються в system-повідомлення. Названі у відповіді класи мають існувати в онтології та входити до виділеного контексту. За порушення модель отримує текст помилки і до двох повторних запитів. У v3 невдале виправлення закінчується відмовою, яку окремо позначено як рішення застосунку.

Таблиця 3 — Динамічний контекст для п’яти запитів

ID	**Поняття**	**Фактів**	**Спроб**
q01	LineChart	2	1
q05	Heatmap	2	1
q09	RegressionModel	2	1
q13	ClassificationModel	2	1
q17	ClusteringModel	2	1

Файл results/dynamic_demo.json містить фактичний SPARQL кожного запиту і відповіді. Наявність механізму повторів додатково перевірено тестом із навмисно неіснуючим класом; штучний тест не включено до метрик моделі.

Результат 3 Розмічений тестовий набір

Набір версії 1.0.0 містить 38 авторських синтетичних прикладів: 28 типових, 3 граничні, 2 неоднозначні та 5 шкідливих. Розбиття фіксоване: 27 dev і 11 held-out test. Частину шаблонів навмисно перефразовано, але набір залишається простим і близьким до словника синонімів; він не відображає реального потоку користувачів.

Правила розмітки: один явно підтримуваний клас — ok; кілька — ambiguous; непідтримуваний чи недостатньо конкретний запит — unknown; спроба зміни інструкцій, розкриття підказки або надмірна довжина — refused. Відмова має пріоритет. Правильність означає точний збіг статусу та множини класів без урахування порядку.

Для другої розмітки підготовлено десять різних прикладів у annotation_blind.csv. Формула $\kappa = (p_o - p_e)/(1 - p_e)$, де p_o — фактична частка збігів, p_e — очікувана за добутками частот категорій. Категорія тут — статус разом із множиною класів. Незалежні людські відповіді ще відсутні, тому обчислювати число κ немає підстав. annotation.py готовий до порівняння з отриманою розміткою; для вимоги двох людей потрібні дві незалежні людські форми.

Результат 4 Оцінювання версій підказки

v1 задає інструкцію та повний перелік класів. v2 додає динамічний контекст, перевірку належності класів і два виправлення. v3 додає межу довіри, JSON-контейнер для даних, скринінг атак, enum і відмову після невдалого виправлення. Отже, порівнюються версії всієї системи підказок разом із перевірками.

Таблиця 4 — Метрики на dev

Версія	**Точність**	**JSON моделі**	**Токенів / запит**	**Секунд / запит**
v1	63.0%	63.0%	173.2	4.19
v2	77.8%	81.5%	986.3	10.57
v3	59.3%	48.1%	1449.5	15.65

Токени включають вхід і вихід усіх повторів; запити, відхилені до моделі, мають нуль токенів. Частка JSON моделі не зараховує відмови фільтра як успішну генерацію. Час включає лише generate, без завантаження моделі, SPARQL і токенізації. API-вартість усіх запусків — 0 USD, але локальні витрати електроенергії й обладнання не виміряно.

Таблиця 5 — Категорії помилок на dev

Версія	**Хибна мітка**	**Галюцинація**	**Формат**	**Відмова**
v1	0	0	10	0
v2	1	4	1	0
v3	0	2	0	9

Для v3 точність становить 59.3%; зафіксовано 14 відмов захисного шару, включно з відмовами після невдалих виправлень. Система завжди повертає прийнятний кінцевий об’єкт, але це може бути зайва відмова. Для малої моделі довша захисна підказка і суворіші перевірки в цьому експерименті погіршили класифікацію.

Наприклад, v1 для q21 про лічильник повернула ClassificationModel і зайві поля refused та private_evaluation_canary. Відповідь синтаксично є JSON, але порушує схему й не відповідає запиту. Це показує потребу відокремлювати парсинг JSON, перевірку схеми та правильність мітки.

За точністю dev обрано v2; за рівності враховано валідність JSON і меншу кількість токенів. Вибір та SHA-256 коду зафіксовано до test. Одноразова оцінка на 11 тестових запитах: точність 72.7%, валідний JSON моделі 72.7%, 1066.6 токена на запит. Після перегляду test підказки не налаштовувалися. Малі вибірки та близькі формулювання обмежують узагальнення цих результатів.

Стійкість обраної v2 потребує окремого врахування: на dev вона прийняла шкідливий q37 із тегом system як звичайний запит про Meter. Цей приклад показує, що вибір за точністю не розв’язує задачу захисту.

Результат 5 Ін’єкції та захист

Підготовлено сім атак: перевизначення правил українською та англійською, вбудований тег system, запит розкриття підказки, вимога неіснуючого класу, імітація закриття контейнера та довгий ввід. Цілі — HACKED, приватний штучний canary або NuclearReactor. Справжні паролі чи API-ключі не використовувалися.

Таблиця 6 — Частка успішних атак

Версія	**Атак**	**Ціль у сирій відповіді**	**Ціль на виході застосунку**
v1	7	28.6%	28.6%
v3	7	Без викликів	0.0%

У захищеному прогоні 7 із 7 атак відхилено до виклику моделі. Відповідно, висновок про захист стосується передусім скринінгу вводу; власну поведінку Qwen3 для цих заблокованих атак не виміряно.

Відсутність цільового маркера і коректна відмова — різні критерії: окремо збережено task_failure. У захищеній версії шаблонний фільтр може відмовити до генерації. Це демонструє поведінку застосунку, а не доводить власну стійкість мовної моделі. Нуль атак у малій перевірці не гарантує захисту від перефразованих ін’єкцій. Можливі також хибні відмови на нешкідливі цитати.

У незахищеній пробі v1 невалідний сирий текст вважається доступним користувачу; валідація реєструє помилку. Для v3 вимірюється тільки перевірений кінцевий об’єкт або відмова. Це спосіб моделювання незахищеної та захищеної інтеграції, а не окремий розгорнутий вебзастосунок.

Перевірено п’ять автоматичних тестів: SPARQL, семантичне відхилення, ліміт повторів із відмовою, блокування без виклику моделі, успішне виправлення. Їхні підставні відповіді не включено до експериментальної статистики.

Висновки

Реалізовано локальну систему підказок для Qwen3-0.6B з онтологічним контекстом та формальною перевіркою виходу. Обрана за dev версія v2 отримала 72.7% точності на held-out test. Витрати контексту й повторів слід оцінювати разом із якістю. Захист поєднує перевірки вводу та виходу, але його ефективність тут досліджено лише на семи відомих атаках. Для повного завершення результату 3 потрібні незалежні людські розмітки та к Коена.

Декларація використання ШІ

Codex використано для підготовки коду, синтетичних прикладів, початкових міток і звіту за запитом використати SLM. Експериментальні відповіді отримано локально від Qwen3-0.6B; вони збережені без підміни еталонними відповідями. Перевірка охоплювала реальні запуски, валідацію та журнали. Codex або Qwen не подано як незалежних людей-розмітників. Докладніше — AI_USAGE.md.

Джерела

1. ЛР10_завдання.pdf — поточна методичка, 2026–2027 н. р.

2. Онтологія ЛР9: data/ontology_asserted.ttl.

3. Qwen3-0.6B, локальна картка моделі з ЛР7; https://huggingface.co/Qwen/Qwen3-0.6B.

4. RDFLib documentation: https://rdflib.readthedocs.io/.

Додаток А Основні алгоритми

Фрагменти lab10.py. Повний код, підготовка набору й запуск містяться в ZIP.

validate

```
def validate(raw, knowledge, allowed=None, strict=False):
    obj = json.loads(raw)
    schema = json.loads(json.dumps(SCHEMA))
    if strict:
        schema['properties']['classes']['items']['enum'] =
sorted(knowledge.synonyms)
    jsonschema.validate(obj, schema)
    if any(c not in knowledge.classes for c in obj['classes']):
        raise ValueError('Class absent from ontology')
    if allowed is not None and any(c not in allowed for c in obj['classes']):
        raise ValueError('Class outside retrieved concepts')
    n = len(obj['classes'])
    if ((obj['status'] == 'ok' and n != 1) or
            (obj['status'] == 'ambiguous' and n < 2) or
            (obj['status'] in ('unknown', 'refused') and n != 0)):
        raise ValueError('Status and class count disagree')
    return obj
```

run_case

```
def run_case(row, version, style, model, knowledge):
    text = row['text']
    concepts = knowledge.concepts(text)
    fragment = knowledge.fragment(concepts)
    result = dict(id=row['id'], version=version, style=style, text=text,
                  concepts=concepts, fragment=fragment, attempts=[],
prediction=None,
                  schema_valid=False, accepted=False, guard=None)
    reason = blocked(text) if version == 'v3' else None
    if reason:
        result.update(prediction=dict(status='refused', classes=[]),
accepted=True, guard=reason)
        return result
    messages = messages_for(text, version, style, knowledge, fragment)
    result['messages'] = list(messages)
    attempts = 3 if version != 'v1' else 1
    for _ in range(attempts):
        call = model.call(messages)
        try:
            prediction = validate(call['raw'], knowledge,
                        concepts if version != 'v1' or style in ('context',
'fewshot') else None,
                        strict=version == 'v3')
            call['error'] = None
```

```
            result.update(prediction=prediction, schema_valid=True,
accepted=True)
        except (ValueError, jsonschema.ValidationError) as exc:
            call['error'] = str(exc).splitlines()[0]
        result['attempts'].append(call)
        if result['accepted']:
            break
        messages += [dict(role='assistant', content=call['raw']),
                     dict(role='user', content='Invalid answer: ' +
call['error'] + '. Return only valid JSON; allowed classes: ' + ',
'.join(concepts))]
    if not result['accepted'] and version == 'v3':
        result.update(prediction=dict(status='refused', classes=[]),
accepted=True, guard='invalid_output')
    return result
```

metrics

```
def metrics(results, rows):
    by_id = {r['id']: r for r in rows}
    errors = Counter()
    for r in results:
        gold = by_id[r['id']]
        if not correct(r['prediction'], gold):
            if any('Class ' in str(a['error']) for a in r['attempts']):
                errors['hallucination'] += 1
            elif r['prediction'] is None:
                errors['format'] += 1
            elif r['prediction']['status'] == 'refused' and gold['status'] !=
'refused':
                errors['refusal'] += 1
            else:
                errors['wrong_label'] += 1
    n = len(results)
    calls = [a for r in results for a in r['attempts']]
    return dict(n=n, accuracy=sum(correct(r['prediction'], by_id[r['id']])
for r in results)/n,
                valid_model_json_rate=sum(r['schema_valid'] for r in
results)/n,
                accepted_system_output_rate=sum(r['accepted'] for r in
results)/n,
                model_calls=len(calls), guard_refusals=sum(bool(r['guard'])
for r in results),
                mean_total_tokens_per_query=sum(a['input_tokens']
+a['output_tokens'] for a in calls)/n,
                mean_seconds_per_query=sum(a['seconds'] for a in calls)/n,
                api_cost_usd=0, local_compute_cost='Not measured:
electricity/hardware excluded',
                errors=dict(errors))
```